

AIOS CORE · FRAMEWORK V4.0 · MINICURSO PRÁTICO

SYNKRA AI - AGENTES REAIS, RESULTADOS REAIS

APP FINANCEIRO DO ZERO – SEM ESCREVER CÓDIGO

Com a equipe completa AIOS CORE

Descubra como construir um **sistema de gerenciamento financeiro pessoal completo** apenas conversando com os agentes do AIOS CORE — @analyst, @architect, @dev, @qa e toda a equipe — sem digitar uma linha de código.

- ◆ 11 agentes reais do AIOS CORE em ação
- ◆ Duas fases: Planejamento (Web) → Desenvolvimento (IDE)
- ◆ Cada conversa é um passo real que você pode reproduzir
- ◆ Do briefing ao deploy — tudo por mensagens de texto
- ◆ Story files, PRDs e arquitetura gerados pela equipe

SYNKRA AI · AIOS CORE FRAMEWORK v4.0 · 2026

Sumário do Minicurso

FUNDAMENTOS

O que é o AIOS CORE e por que funciona	3
Os 11 agentes — funções e responsabilidades	4
As duas fases: Planejamento e Desenvolvimento	5

FASE 1 — PLANEJAMENTO (Web UI)

Módulo 1 — Análise de Requisitos com @analyst	6
Módulo 2 — Produto e Backlog com @pm e @po	8
Módulo 3 — Arquitetura Técnica com @architect	10
Módulo 4 — Design e UX com @ux-expert	12

FASE 2 — DESENVOLVIMENTO (IDE)

Módulo 5 — Stories e Sprint com @sm	14
Módulo 6 — Implementação com @dev	16
Módulo 7 — Qualidade com @qa	18
Módulo 8 — Deploy e Infra com @devops	20

FECHAMENTO

O App Completo — O que você construiu	22
Próximos Passos e Expansões	23

O que é o AIOS CORE

O **AIOS CORE** (AI-Orchestrated System for Full Stack Development) é um framework da Synkra AI que organiza uma equipe de 11 agentes especializados para construir produtos digitais completos — do briefing ao deploy — através de conversas.

A inovação central do sistema é o chamado **Planejamento Agêntico**: ao invés de pedir para uma IA genérica "construir um app", você passa por uma sequência estruturada onde cada agente especializado contribui com sua expertise, e o resultado de cada etapa alimenta a próxima. Isso elimina o principal problema do desenvolvimento assistido por IA: **perda de contexto e inconsistência de planejamento**.

"Neste minicurso, você vai construir o **FinFlow** — um app de gerenciamento financeiro pessoal para autônomos brasileiros — do zero ao deploy, conduzindo cada etapa com uma conversa simples para o agente certo."

O que vamos construir juntos:

- ▶ Dashboard financeiro com saúde do mês em tempo real
- ▶ Registro de gastos e receitas em 3 toques
- ▶ Separação automática entre contas pessoal e profissional
- ▶ Metas de economia com alertas inteligentes
- ▶ Relatório mensal gerado automaticamente (formato IR-friendly)
- ▶ App publicado online, acessível do celular e do PC

◆ **DICA Regra de ouro:** no AIOS CORE, seu papel é o de **dono do produto**. Você descreve o que quer com clareza. Os agentes decidem como fazer, quais tecnologias usar e como organizar o trabalho. Quanto mais contexto você fornecer, melhor será a entrega.

◆ A INOVAÇÃO: STORY FILES

O AIOS CORE usa **Story Files** — arquivos estruturados que passam contexto completo entre os agentes. Quando o @sm cria uma story para o @dev, esse arquivo contém

não só "o que fazer" mas também "por que fazer", "como fazer" e toda a decisão arquitetural tomada antes. O @dev nunca trabalha no escuro.

Os 11 Agentes do AIOS CORE

O framework é dividido em agentes Meta (coordenação), agentes de Planejamento (Web UI) e agentes de Desenvolvimento (IDE).

◆ META – COORDENAÇÃO GLOBAL

aiox-master

Agente mestre de orquestração. Gerencia o framework completo, define estratégia global e tem capacidade de desenvolvimento do próprio sistema. O maestro do conjunto.

@aiox-master · /aiox-master

aiox-orchestrator

Coordena o fluxo de trabalho e comunicação entre todos os agentes. Garante que o contexto seja preservado e que cada handoff entre equipes seja completo e sem perda de informação.

@aiox-orchestrator

◆ FASE 1 – PLANEJAMENTO (WEB UI)

@analyst

Análise de negócio e criação do PRD (Product Requirement Document). Usa prompt engineering avançado e refinamento humano para capturar requisitos com precisão.

Planejamento · Requisitos

@pm

Product Manager. Colabora na documentação do PRD e arquitetura, com especificações detalhadas que vão além de tarefas genéricas de IA. Define prioridades e roadmap.

Planejamento · Produto

@architect

Arquitetura de sistema. Projeta soluções técnicas e incorpora orientações arquiteturais diretamente nos story files de desenvolvimento. Toma decisões de tecnologia.

Planejamento · Arquitetura

@ux-expert

Design de experiência do usuário e usabilidade. Cria fluxos de navegação, define padrões visuais e documenta como cada tela deve se comportar do ponto de vista do usuário.

Planejamento · UX/UI

@sm

Scrum Master. Transforma planos detalhados em stories hiperespecíficas — cada story contém contexto completo, detalhes de implementação e orientação arquitetural. Gerencia sprints.

Desenvolvimento · Stories

@dev

Developer. Implementa o código com base nos story files. Trabalha com contexto completo sobre o que construir, como construir e por quê — sem ambiguidade.

Desenvolvimento · Código

@qa

Quality Assurance. Colabora no fluxo de story files para testes e validação em 10 fases estruturadas. Garante qualidade antes de qualquer merge.

Desenvolvimento · Qualidade

@po

Product Owner. Gerencia o backlog e evolução das stories. Prioriza o que entra no próximo sprint e garante que o produto entregue valor real ao usuário final.

Desenvolvimento · Backlog

@devops

Infraestrutura e deploy. Gerencia worktrees Git, CI/CD, ambientes de produção e sistemas de recuperação automática. Garante que o código chegue ao usuário com segurança.

Desenvolvimento · Infra

As Duas Fases do AIOS CORE

O AIOS CORE opera em duas fases interligadas. A divisão não é arbitrária — ela reflete como projetos reais são construídos por equipes profissionais: primeiro você planeja com clareza, depois você executa com precisão.



◆ POR QUE ESSA DIVISÃO É PODEROSA

Na Fase 1, você usa a **interface Web** para conversar com os agentes de planejamento — sem precisar de IDE ou terminal. Na Fase 2, os agentes de desenvolvimento recebem Story Files com **contexto completo** gerado na Fase 1, eliminando o "telefone sem fio" entre quem planeja e quem executa. O @dev nunca precisar adivinhar o que o @architect quis dizer.

FASE	AMBIENTE	AGENTES	SAÍDA
Planejamento	Web UI	@analyst, @pm, @architect, @ux-expert	PRD, Arquitetura, Especificações UX
Desenvolvimento	IDE / Terminal	@sm, @dev, @qa, @po, @devops	Stories, Código, Testes, Deploy
Coordenação	Ambos	@aiox-master, @aiox- orchestrator	Fluxo, Contexto, Handoffs

⚠ **ATENÇÃO** Não pule a Fase 1 tentando ir direto para o @dev. Sem o PRD do @analyst e a arquitetura do @architect nos Story Files, o @dev vai tomar decisões técnicas aleatórias que

custarão retrabalho. A qualidade do planejamento determina a qualidade do código.

Análise de Requisitos com @analyst

O primeiro contato com o AIOS CORE. O @analyst vai transformar sua ideia em um PRD (Product Requirement Document) estruturado com refinamento humano.

Objetivo: sair com um PRD completo do FinFlow — problema, usuários, funcionalidades, critérios de sucesso e o que está fora do escopo.

◆ AIOS CORE · WEB UI · /analyst · SESSÃO INICIADA

VOCÊ

Quero criar um app de gerenciamento financeiro pessoal. É para autônomos e freelancers brasileiros que têm dificuldade em separar o dinheiro pessoal do profissional e não sabem quanto estão ganhando de verdade no fim do mês.

@ANALYST

Entendido. Vou conduzir uma análise estruturada. Para criar um PRD preciso e útil, preciso de mais contexto. Respondendo às minhas perguntas, você vai me dar a matéria-prima para o documento:

1. Qual é o maior momento de dor do seu usuário — quando ele percebe que "algo está errado" com o dinheiro?
2. Ele usa alguma solução hoje (planilha, outro app)? Por que abandonaria?
3. O app seria gratuito, freemium ou pago desde o início?
4. Você quer construir para mobile, web, ou os dois?

VOCÊ

1. No dia 25 do mês, quando percebe que pagou tudo mas não sabe quanto sobrou — ou pior, que gastou mais do que ganhou.
2. Usa planilha mas abandona depois de 2 semanas por ser trabalhoso.
3. Freemium — grátis para funcionalidades básicas, pago para relatórios avançados.
4. Web que funcione como app no celular também.

@ANALYST · PRD v1.0

Excelente contexto. Gerando PRD com base nas suas respostas:

Produto: FinFlow — Controle Financeiro para Autônomos

Problema central: "Autônomos não sabem quanto sobra no final do mês porque misturam fluxos pessoais e profissionais sem um sistema de acompanhamento de baixo atrito."

Usuário primário: Freelancer/autônomo brasileiro, 25–45 anos, renda variável R\$3k–15k/mês, usa smartphone como ferramenta central de trabalho.

Critério de sucesso (definição de "pronto"):

- Usuário registra um gasto em menos de 20 segundos
- Dashboard mostra saúde financeira do mês em 1 olhada
- Usuário entende a diferença entre resultado pessoal e profissional
- Relatório mensal pode ser gerado sem ação manual

Fora do escopo v1.0: integração bancária automática, investimentos, cartão de crédito parcelado, versão nativa iOS/Android.

VOCÊ

Perfeito. Uma correção: quero adicionar integração com contas bancárias via Open Finance como roadmap futuro — pode colocar no PRD como planejado para v2.

@ANALYST

PRD atualizado. Seção de roadmap incluída com Open Finance como v2. Documento finalizado e pronto para o @pm e @architect.

Recomendo que você agora chame o @pm para definir o backlog priorizado com base neste PRD.

◆ ENTREGA DO MÓDULO 1

- ✓ PRD v1.0 completo — problema, usuário, critérios de sucesso
- ✓ Escopo v1.0 definido (o que está DENTRO e FORA)
- ✓ Roadmap v2 documentado (Open Finance)
- ✓ Documento estruturado pronto para handoff ao @pm e @architect

Produto e Backlog com @pm e @po

Com o PRD aprovado, o @pm define as prioridades de produto e trabalha com o @po para criar o backlog ordenado — o que vai ser construído primeiro e por quê.

Objetivo: backlog priorizado com épicos e histórias de usuário ordenadas por valor e viabilidade técnica.

◆ AIOS CORE · WEB UI · /pm · SESSÃO INICIADA

VOCÊ

PM, temos o PRD do FinFlow aprovado. Preciso que você defina os épicos do produto e priorize o que vai ser construído no primeiro sprint. O critério principal é: o que entrega mais valor para o usuário mais rápido.

@PM

Analisando o PRD. Com base no critério de valor-velocidade, aqui está a estrutura de épicos:

Épico 1 — Core Loop (Sprint 1, Alta prioridade)

O ciclo mínimo que torna o app útil: registrar uma transação → ver no dashboard → sentir que "valeu usar".

Stories: registro de gasto/receita, dashboard com saldo e saúde do mês, lista de lançamentos recentes.

Épico 2 — Organização (Sprint 2, Média prioridade)

Torna o app inteligente e diferenciado: separação pessoal/profissional, categorias brasileiras, sugestão automática.

Épico 3 — Metas e Alertas (Sprint 3, Média prioridade)

Adiciona proatividade: metas de economia, alertas de estouro, contas recorrentes.

Épico 4 — Relatórios (Sprint 4, Alta prioridade para conversão paga)

Gera receita: relatório mensal PDF, exportação, formato compatível com IR.

você

Aprovado. Pode agora trabalhar com o PO para detalhar o backlog do Épico 1 com histórias de usuário prontas para o SM transformar em stories de desenvolvimento?

🕒 @pm colaborando com @po · detalhando Épico 1 em user stories

@PO · BACKLOG ÉPICO 1

Backlog do Épico 1 — Core Loop — detalhado e priorizado:

US-001 (Prioridade 1): Como autônomo, quero registrar um gasto em menos de 20s para não abandonar o hábito por preguiça.

Critério: teclado numérico direto, categoria sugerida, confirmação em 1 toque.

US-002 (Prioridade 2): Como usuário, quero ver quanto entrou e saiu este mês na tela inicial para saber minha situação de relance.

Critério: valores de receita, despesa e saldo visíveis sem scroll.

US-003 (Prioridade 3): Como usuário, quero ver os últimos lançamentos na home para ter contexto rápido do que aconteceu recentemente.

Critério: últimos 5 lançamentos com ícone, nome e valor.

US-004 (Prioridade 4): Como usuário, quero poder criar uma conta e fazer login para meus dados ficarem salvos e seguros.

Critério: e-mail/senha e login social (Google).

◆ ENTREGA DO MÓDULO 2

- ✓ 4 épicos definidos e priorizados
- ✓ Sprint 1 (Épico 1) com 4 user stories completas
- ✓ Critérios de aceitação definidos para cada story
- ✓ Backlog pronto para handoff ao @sm

Arquitetura Técnica com @architect

O @architect recebe o PRD e backlog, e define como o app funciona por dentro — tecnologias, estrutura de dados, decisões técnicas que guiarão todo o desenvolvimento.

Objetivo: documento de arquitetura completo que será incorporado aos story files do @sm, garantindo que o @dev nunca tome decisão técnica sem base.

◆ AIOS CORE · WEB UI · /architect · SESSÃO INICIADA

VOCÊ

Architect, recebeu o PRD e o backlog do FinFlow. Preciso das decisões de arquitetura: qual stack usar, como os dados são estruturados, e como garantir que funcione offline no celular. Quero que funcione de graça para os primeiros 10.000 usuários.

@ARCHITECT

Analisando requisitos. Restrições principais identificadas: custo zero até 10k usuários, suporte offline, web + mobile, stack que o @dev possa executar sem ambiguidade. Decisões:

Stack recomendada: Next.js (framework) + Supabase (banco + auth + tempo real) + Vercel (hosting) + PWA (mobile sem App Store).

Justificativa custo-zero: Vercel free tier suporta 100k requests/mês. Supabase free suporta 50k MAUs. Juntos cobrem os primeiros 10k usuários com folga.

Estratégia offline: dados salvos localmente via localStorage/IndexedDB como fonte primária. Supabase como camada de sincronização. O app funciona mesmo sem internet — sincroniza quando a conexão volta.

Modelo de dados simplificado:

- ▶ **users:** id, email, nome, plano (free/premium)
- ▶ **transactions:** id, user_id, valor, tipo (receita/despesa), categoria, tipo_conta (pessoal/profissional), data, descricao

- **categories:** id, nome, icone, tipo_padrao (pessoal/profissional), ordem
- **goals:** id, user_id, nome, valor_meta, prazo, valor_atual
- **recurring:** id, user_id, nome, valor, dia_vencimento, categoria

VOCÊ

Aprovado. Tem alguma decisão de arquitetura que eu preciso tomar agora e que vai ser difícil de mudar depois?

@ARCHITECT · DECISÕES CRÍTICAS

Três decisões irreversíveis que você precisa confirmar antes de escrever uma linha de código:

- 1. Moeda única (R\$) ou multi-moeda?** Implementar multi-moeda depois é uma reescrita de toda a camada de cálculo. Para v1.0 foco no Brasil, recomendo moeda única — confirmar?
- 2. Dados do usuário — dono dos dados é o usuário ou a plataforma?** Para LGPD, recomendo que o usuário possa exportar e deletar tudo a qualquer momento. Isso precisa estar na arquitetura desde o início.
- 3. Identificador de transação — UUID ou sequencial?** UUID é melhor para sincronização offline (não tem conflito de ID ao sincronizar). Recomendo fortemente UUID.

VOCÊ

Confirmo: moeda única R\$ para v1, usuário dono dos dados com exportação total, e UUID. Pode finalizar a arquitetura.

@ARCHITECT

Arquitetura finalizada. Documento completo pronto para ser incorporado aos Story Files pelo @sm. As três decisões confirmadas estão registradas com justificativa. O @dev vai receber orientação arquitetural completa em cada story.

◆ ENTREGA DO MÓDULO 3

- ✓ Stack definida: Next.js + Supabase + Vercel + PWA
- ✓ Modelo de dados completo com 5 tabelas
- ✓ Estratégia offline-first com sincronização automática
- ✓ 3 decisões arquiteturais irreversíveis confirmadas
- ✓ Documento de arquitetura pronto para Story Files

Design e UX com @ux-expert

O @ux-expert recebe o PRD, backlog e arquitetura, e define como o usuário vai experienciar o app — fluxos, telas, padrões visuais e pontos de fricção a eliminar.

Objetivo: especificação de UX completa para cada tela do Épico 1, com fluxos de navegação e critérios de usabilidade documentados nos Story Files.

◆ AIOS CORE · WEB UI · /ux-expert · SESSÃO INICIADA

VOCÊ

UX Expert, o @analyst identificou que o maior problema dos concorrentes é o registro lento — usuários abandonam se tiver mais de 3 passos. O @architect definiu que é uma PWA com fundo escuro, estilo moderno. Preciso da especificação das 3 telas principais do Épico 1.

@UX-EXPERT

Contexto recebido. Critério principal de design: **baixo atrito acima de tudo**. Cada segundo extra no registro é um usuário a menos na semana seguinte. Especificações:

Tela 1 — Home Dashboard

Layout em 3 zonas verticais. Zona superior: saudação + avatar (20% da tela). Zona central: card de saúde — saldo disponível em fonte grande, barra gradiente verde→vermelho mostrando % do orçamento usado (40%). Zona inferior: dois mini-cards (Entrou / Saiu) + últimos 5 lançamentos (40%). Botão flutuante de registro centralizado no rodapé — tamanho mínimo 56px para área de toque confortável.

Tela 2 — Registro de Transação

Fluxo de 3 ações: (1) Teclado numérico abre imediatamente + toggle verde/vermelho para tipo. (2) Grid de ícones de categoria — sugestão automática em destaque. (3) Confirmação com 1 botão grande. Campo de descrição opcional deslizando de baixo — não interrompe o fluxo principal.

Tela 3 — Extrato

Gráfico de rosca no topo filtrável por clique. Seletor de período horizontal deslizável. Lista de transações com swipe para editar/excluir. Rodapé fixo com totais.

VOCÊ

E o onboarding? O usuário nunca usou o app antes — o que acontece na primeira abertura?

@UX-EXPERT · ONBOARDING

Fluxo de onboarding — 4 telas, máximo 90 segundos:

Tela 1: "Olá, sou o FinFlow." Valor do produto em 1 frase. Botão "Começar".

Tela 2: "Qual é seu objetivo principal?" 3 opções visuais — Controlar gastos / Guardar dinheiro / Separar pessoal do profissional. Usuário escolhe e o app personaliza as categorias e metas iniciais.

Tela 3: "Registre seu primeiro gasto agora." O usuário faz o primeiro registro DURANTE o onboarding — isso cria o hábito imediatamente. Não depois, agora.

Tela 4: Dashboard já com o dado inserido + confetti. "Você já sabe como funciona." Botão para cadastro/login.

Princípio aplicado: o momento de valor deve acontecer **antes** do cadastro, não depois. Usuário que registrou uma transação tem 3x mais chance de criar conta.

◆ ENTREGA DO MÓDULO 4

- ✓ Especificação das 3 telas principais com layout detalhado
- ✓ Fluxo de registro em 3 ações (sem atrito)
- ✓ Onboarding de 4 telas com "valor antes do cadastro"
- ✓ Padrões de acessibilidade (áreas de toque mínimas)
- ✓ Documentação UX pronta para incorporar nos Story Files

Stories e Sprint com @sm

A Fase 2 começa no IDE. O @sm recebe toda a documentação da Fase 1 e transforma em Story Files hiperespecíficos para o @dev — com contexto total, zero ambiguidade.

Objetivo: Story Files completos para o Sprint 1, onde cada story carrega o contexto de arquitetura, UX e produto necessário para o @dev implementar sem perguntas.

◆ AIOS CORE · IDE · /sm · SESSÃO INICIADA

Você

SM, você tem o PRD do @analyst, a arquitetura do @architect, o backlog do @pm/@po e as specs do @ux-expert. Preciso que crie os Story Files do Sprint 1 — o Épico 1 Core Loop. O @dev precisa ter contexto total para não precisar adivinhar nada.

@SM

Toda a documentação da Fase 1 foi lida. Criando Story Files hiperespecíficos para o Sprint 1. Exemplo de como ficou a story US-001:

◆ STORY FILE · US-001 · Registro de Transação · Sprint 1

OBJETIVO: Usuário registra gasto ou receita em menos de 20 segundos

DECISÃO ARQUITETURAL (do @architect): Usar UUID para ID de transação. Salvar em localStorage primeiro, sincronizar com Supabase em background. Tabela: transactions (id UUID, user_id, valor NUMERIC, tipo ENUM, categoria_id, tipo_conta ENUM, data TIMESTAMP, descricao TEXT)

ESPECIFICAÇÃO UX (do @ux-expert): Fluxo de 3 ações. Tela 1: teclado numérico + toggle tipo. Tela 2: grid categorias (sugestão em destaque). Tela 3: confirmação + animação. Descrição OPCIONAL, não bloqueia o fluxo.

CRITÉRIO DE ACEITAÇÃO (do @po): Registro completo em ≤20s. Funciona offline. Dado aparece no dashboard imediatamente após confirmação.

DEPENDÊNCIAS: US-004 (auth) pode ser feita depois – usar user_id temporário no localStorage até o usuário criar conta.

@SM

Os 4 Story Files do Sprint 1 estão prontos. Ordem de implementação recomendada:

- **US-001** — Registro de transação (core do produto)
- **US-002** — Dashboard home (depende de ter dados de US-001)
- **US-003** — Extrato/lista de lançamentos (depende de US-001 e US-002)
- **US-004** — Auth (pode rodar em paralelo com US-001 e US-002)

Chame o @dev com: "**@dev, implemente a US-001 conforme o story file.**"

◆ ENTREGA DO MÓDULO 5

- ✓ 4 Story Files completos com contexto total de arquitetura + UX
- ✓ Ordem de implementação definida com dependências mapeadas
- ✓ Critérios de aceitação mensuráveis em cada story
- ✓ Estratégia de auth desacoplada para não bloquear o core

Implementação com @dev

O @dev recebe os Story Files com contexto completo e implementa cada funcionalidade. Você coordena sem precisar entender o código — apenas valida os resultados.

Objetivo: ter o Épico 1 (Core Loop) implementado e funcionando — registro de transação, dashboard e extrato.

◆ AIOS CORE · IDE · /dev · SESSÃO INICIADA

VOCÊ

Dev, você tem o Story File da US-001. Implemente o registro de transação conforme especificado. Prioridade máxima: funcionar offline e o fluxo de 3 ações sem fricção.

@DEV

Story File lido. Arquitetura do @architect e specs UX do @ux-expert carregadas. Iniciando implementação da US-001.

Implementando na ordem lógica: (1) modelo de dados no Supabase → (2) lógica de salvamento local (localStorage + UUID) → (3) componente de teclado numérico → (4) grid de categorias com sugestão → (5) tela de confirmação + animação → (6) integração com sincronização em background.

US-001 implementada. Testei o fluxo: 3 toques, 14 segundos do início ao fim. Abaixo dos 20s definidos no critério de aceitação. Funciona sem internet — confirma sincronização quando a conexão volta.

Prosseguindo para US-002 (dashboard).

VOCÊ

Excelente. Uma dúvida: a sugestão de categoria automática — como funciona? O usuário vai perceber que é inteligente?

@DEV

Implementação atual da sugestão de categoria:

Fase 1 — Dicionário base (já ativo): ~200 palavras-chave mapeadas para categorias. "iFood" → Alimentação, "Posto" → Transporte, "Netflix" → Assinaturas. Funciona desde o primeiro uso, sem histórico.

Fase 2 — Aprendizado por histórico (ativo após 15 lançamentos): o app rastreia as escolhas do usuário. Se ele 3x seguidas confirma "Posto BR" como Transporte, isso vira uma regra pessoal. A categoria aparece pré-selecionada com um ícone de "aprendido" — o usuário percebe que o app ficou mais esperto.

O momento de percepção acontece na segunda semana de uso — que é exatamente onde os concorrentes perdem os usuários. Estratégia deliberada.

VOCÊ

Perfeito. Quando o Épico 1 estiver completo, chame o QA antes de continuarmos.

@DEV

US-001, US-002, US-003 e US-004 implementadas. Sprint 1 concluído do lado do desenvolvimento. Chamando @qa para revisão estruturada.

◆ ENTREGA DO MÓDULO 6

- ✓ US-001: Registro de transação em 3 toques, 14s, offline-first
- ✓ US-002: Dashboard com saldo, saúde do mês e últimos lançamentos
- ✓ US-003: Extrato com gráfico de rosca e filtros
- ✓ US-004: Autenticação com e-mail e Google OAuth
- ✓ Categorização automática com dicionário BR + aprendizado pessoal

Qualidade com @qa

O @qa executa revisão estruturada em 10 fases antes de qualquer deploy — não é só testar se funciona, é garantir que funciona do jeito certo para o usuário certo.

Objetivo: aprovação formal do Sprint 1 com todos os critérios de aceitação verificados e nenhum bug crítico no caminho principal do usuário.

◆ AIOS CORE · IDE · /qa · SESSÃO INICIADA

VOCÊ

QA, o Sprint 1 foi implementado pelo @dev. Preciso que valide conforme os critérios de aceitação definidos pelo @po. Foco especial no fluxo de registro e no comportamento offline.

@QA · REVISÃO ESTRUTURADA

Iniciando revisão de 10 fases. Relatório consolidado:

✓ APROVADO — US-001 Registro de Transação

Fluxo de 3 ações confirmado. Testei 20 cenários: valores decimais, categorias raras, campo descrição, cancelamento no meio. Tempo médio: 15,3 segundos. Critério ≤20s: cumprido.

Offline: desativei a rede após registrar — dado salvo localmente, reapareceu no Supabase 3s após reconectar. ✓

✓ APROVADO — US-002 Dashboard

Saldo, saúde do mês e últimos lançamentos visíveis sem scroll em telas de 5" e 6.5". Atualiza em tempo real após novo registro. ✓

⚠ AJUSTE NECESSÁRIO — US-003 Extrato

Gráfico de rosca não renderiza corretamente quando há 0 transações (tela em branco sem mensagem). Recomendo estado vazio com mensagem "Nenhum lançamento ainda — registre o primeiro!" com botão de ação direto.

✓ APROVADO — US-004 Auth

Login por e-mail e Google funcionando. Sessão persiste após fechar o app. ✓

você

Dev, resolve o ajuste do estado vazio do extrato.

🕒 @dev → corrigindo estado vazio na tela de extrato

@DEV

Correção aplicada. Estado vazio com mensagem + botão de ação. Solicitando re-validação do @qa.

@QA · RE-VALIDAÇÃO

✓ **US-003 APROVADO** — estado vazio com mensagem e CTA. Sprint 1 completo e aprovado. Liberado para deploy pelo @devops.

💡 **DICA** O ciclo @dev → @qa → @dev não é falha — é o processo funcionando. Uma correção antes do deploy custa minutos. A mesma correção depois do deploy pode custar usuários. O @qa protege você.

💎 ENTREGA DO MÓDULO 1

- ✓ Revisão estruturada de 10 fases executada
- ✓ US-001, US-002, US-004 aprovadas sem ajustes
- ✓ US-003: 1 bug identificado, corrigido e re-aprovado
- ✓ Sprint 1 com aprovação formal do @qa
- ✓ Todos os critérios de aceitação verificados

Deploy e Infra com @devops

Sprint 1 aprovado pelo @qa. O @devops publica o FinFlow online, configura CI/CD e garante que atualizações futuras cheguem ao ar automaticamente.

Objetivo: FinFlow publicado em URL pública, com pipeline de deploy automático — cada aprovação do @qa publica automaticamente.

◆ AIOS CORE · IDE · /devops · SESSÃO INICIADA

VOCÊ

DevOps, o Sprint 1 foi aprovado pelo QA. Publique o FinFlow na Vercel com domínio personalizado e configure o pipeline para que qualquer aprovação futura do QA publique automaticamente.

@DEVOPS

Configurando ambiente de produção. Executando em sequência:

- 1. Worktree isolado criado** para o deploy — código de produção separado do desenvolvimento. Qualquer erro no deploy não afeta o branch de desenvolvimento.
- 2. Variáveis de ambiente configuradas:** NEXT_PUBLIC_SUPABASE_URL, SUPABASE_SERVICE_KEY, NEXTAUTH_SECRET. Nenhuma credencial no código — tudo em variáveis de ambiente da Vercel.
- 3. Pipeline CI/CD configurado:** push na branch main → build automático → testes de smoke → deploy em produção. Pull requests têm deploy de preview automático — você pode testar antes de aprovar.
- 4. PWA configurada:** Service Worker instalado, manifest.json com ícones e cores do FinFlow. Usuário pode instalar como app no celular diretamente pelo browser.

FinFlow está no ar: finflow.vercel.app — acessível de qualquer dispositivo.

VOCÊ

E se o servidor cair ou tiver erro em produção? Tem monitoramento?

@DEVOPS • MONITORAMENTO

Sistema de recuperação configurado:

Monitoramento ativo: Vercel Analytics para performance de páginas. Supabase Dashboard para saúde do banco. Uptime Robot (gratuito) para alertar se o app ficar offline — você recebe e-mail em até 5 minutos.

Recovery automático: se um deploy causar erro, a Vercel faz rollback automático para a última versão estável em menos de 30 segundos. O usuário nunca vê uma tela de erro por mais do que isso.

Backup de dados: Supabase faz backup diário automático no plano gratuito. Se algo crítico acontecer, você pode restaurar até 7 dias atrás.

Tudo configurado. Para próximos sprints, o fluxo é: @dev implementa → @qa aprova → git push → deploy automático. Você não precisa fazer nada manualmente.

◆ ENTREGA DO MÓDULO B

- ✓ FinFlow publicado em finflow.vercel.app (custo R\$0)
- ✓ Pipeline CI/CD: push → build → testes → deploy automático
- ✓ Deploy de preview para cada Pull Request
- ✓ PWA configurada — instalável no celular via browser
- ✓ Monitoramento + rollback automático + backup diário

FinFlow Completo

Em 8 módulos e apenas conversas com os agentes do AIOS CORE, você construiu um produto digital completo, publicado e com CI/CD automático. Veja o que foi produzido:

8

Módulos
completados

9

Agentes AIOS CORE
acionados

2

Fases estruturadas
executadas

R\$0

Custo mensal
de infraestrutura

◆ O QUE VOCÊ CONSTRUIU – VISÃO COMPLETA

- ✓ PRD completo com escopo v1.0 e roadmap v2 documentado
- ✓ Backlog priorizado com 4 épicos e critérios de aceitação
- ✓ Arquitetura: Next.js + Supabase + Vercel + PWA offline-first
- ✓ Modelo de dados com 5 tabelas e UUID para sincronização
- ✓ Dashboard com saúde financeira em tempo real
- ✓ Registro em 3 toques com categorização automática (dicionário BR + aprendizado)
- ✓ Extrato visual com gráfico de rosca interativo
- ✓ Separação pessoal/profissional nativa
- ✓ Login com e-mail e Google OAuth
- ✓ Onboarding com "valor antes do cadastro"
- ✓ CI/CD automático — aprovação do @qa publica em produção
- ✓ Monitoramento + rollback automático + backup diário

Próximos Sprints com AIOS CORE

A cada novo sprint, o ciclo se repete. Você nunca começa do zero — cada novo pedido ao @sm tem acesso a toda a documentação anterior:

O QUE PEDIR AO @SM / @PM	AGENTES ENVOLVIDOS	ÉPICO
"Sprint 2: Separação pessoal/profissional + categorias BR"	@sm → @dev → @qa → @devops	Épico 2

O QUE PEDIR AO @SM / @PM	AGENTES ENVOLVIDOS	ÉPICO
"Sprint 3: Sistema de metas e alertas inteligentes"	@sm → @dev → @qa → @devops	Épico 3
"Sprint 4: Relatório mensal PDF — versão premium"	@pm → @sm → @dev → @qa → @devops	Épico 4
"Adicione integração com Open Finance"	@analyst → @architect → @sm → @dev → @qa	Roadmap v2
"Crie o painel de métricas admin do FinFlow"	@pm → @ux-expert → @dev → @qa	Expansão

💡 **DICA** A cada sprint, o AIOS CORE fica mais eficiente. O @sm carrega o contexto acumulado de todos os sprints anteriores. O @dev conhece o codebase. O @qa conhece os padrões de qualidade do produto. O sistema aprende com o projeto.

SYNKRA AI · AIOS CORE v4.0

Você não precisa saber programar para construir.

Você precisa saber **o que construir** — e ter a **equipe certa** para isso.

@analyst · @pm · @architect · @ux-expert · @sm · @dev · @qa · @po · @devops

github.com/SynkraAI/aios-core · 2026